



Instituto Tecnológico de Costa Rica

Escuela de Computación

IC-4700 Lenguajes de Programación

Proyecto 2: Programación Funcional

Docente:

Bryan Tomas Hernandez Sibaja

Estudiantes:

- **Gustavo Pacheco Morales 2022255539**
- **Fernando Abarca Aguilar 2022437534**
- **Jorge Guevara Chavarria 2022437473**

-

Fecha:

12/05/2026

Índice

Enlace de GitHub.....	1
Pasos de instalación del programa.....	1
Manual de usuario.....	2
Arquitectura lógica utilizada.....	5
Explicación del funcionamiento.....	10
Bibliografía.....	11

Enlace de GitHub

https://github.com/ferabarca1910/Lenguajes_Proyectoll#

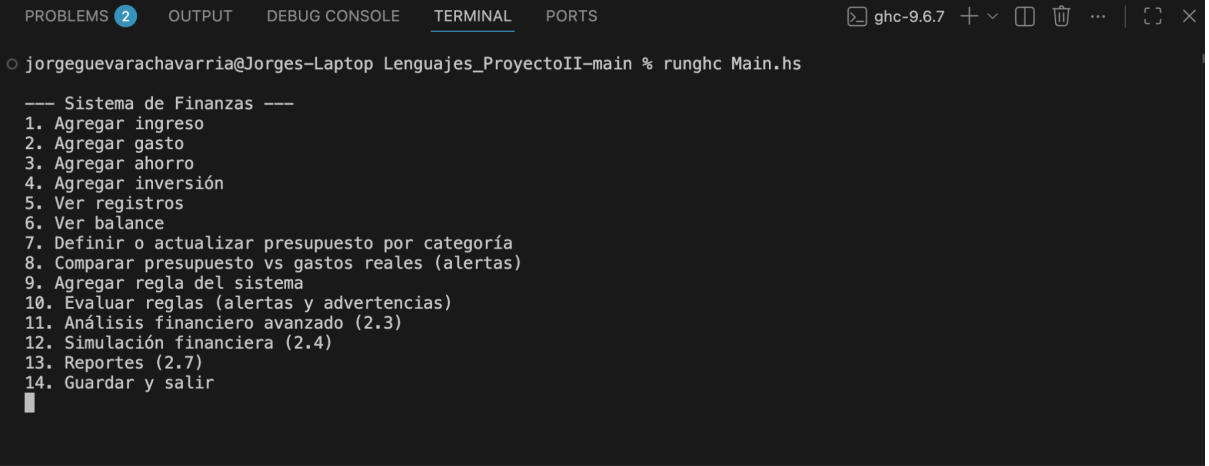
Pasos de instalación del programa

1. Clonar el repositorio
Descargar el proyecto desde GitHub:
git clone https://github.com/ferabarca1910/Lenguajes_Proyectoll.git
2. Abrir una terminal en la carpeta del proyecto
3. Compilar el código:
ghc --make Main.hs -o Main.exe
4. Ejecutar:
.Main.exe
Esto inicia el sistema
5. Requisitos:
Windows 10/11
GHC (Haskell) instalado - [Installation - GHCup](#)
Recomendado: GHCup

Manual de usuario

El sistema funciona mediante un menú interactivo en consola, donde el usuario puede seleccionar diferentes opciones para registrar y analizar información financiera.

Al ejecutar el programa, se muestra el siguiente menú principal:



```
PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS ghc-9.6.7 + - [] ... | [] X
jorgeguevarachavarria@Jorges-Laptop Lenguajes_ProyectoII-main % runghc Main.hs

--- Sistema de Finanzas ---
1. Agregar ingreso
2. Agregar gasto
3. Agregar ahorro
4. Agregar inversión
5. Ver registros
6. Ver balance
7. Definir o actualizar presupuesto por categoría
8. Comparar presupuesto vs gastos reales (alertas)
9. Agregar regla del sistema
10. Evaluar reglas (alertas y advertencias)
11. Análisis financiero avanzado (2.3)
12. Simulación financiera (2.4)
13. Reportes (2.7)
14. Guardar y salir
█
```

El menú cuenta con las siguientes funcionalidades:

1. Agregar ingreso

Permite registrar un ingreso económico indicando:

- monto
- categoría
- fecha
- descripción
- etiquetas

2. Agregar gasto

Permite registrar gastos realizados por el usuario utilizando la misma estructura de información financiera.

3. Agregar ahorro

Registra movimientos financieros asociados a ahorro.

4. Agregar inversión

Permite almacenar registros financieros relacionados con inversiones.

5. Ver registros

Muestra todos los registros almacenados en el sistema con su información detallada.

6. Ver balance

Calcula y muestra el balance total del sistema utilizando la lógica funcional implementada.

7. Definir o actualizar presupuesto por categoría

Permite crear o modificar presupuestos máximos para distintas categorías financieras.

8. Comparar presupuesto vs gastos reales

Compara los gastos registrados contra los presupuestos definidos y genera alertas si se exceden los límites establecidos.

9. Agregar regla del sistema

Permite agregar reglas financieras personalizadas, como límites de gasto o mínimos de ahorro.

10. Evaluar reglas

Analiza los registros financieros y muestra alertas o advertencias cuando alguna regla configurada se cumple.

11. Análisis financiero avanzado

Genera análisis como:

- flujo mensual de caja
- tendencias de gasto
- proyección de gastos
- categorías con mayor impacto financiero

12. Simulación financiera

Permite simular:

- reducción porcentual de gastos
- proyección de ahorro en el tiempo

13. Reportes

Genera reportes financieros como:

- resumen mensual
- comparación entre periodos
- categorías con mayor gasto

14. Guardar y salir

Guarda toda la información del sistema en archivos de texto y finaliza la ejecución del programa.

Arquitectura lógica utilizada

El sistema fue desarrollado utilizando una arquitectura modular basada en el paradigma funcional de Haskell. El proyecto se dividió en varios módulos independientes, permitiendo separar la lógica del sistema, la persistencia de datos, la definición de tipos y la interacción con el usuario.

La arquitectura utilizada se divide en los siguientes módulos:

Types.hs

Contiene la definición de todos los tipos de datos utilizados en el sistema, incluyendo:

- registros financieros
- presupuestos
- reglas del sistema

Se utilizaron tipos algebraicos y registros (**record syntax**) para modelar la información financiera de manera estructurada.

```
1  module Types where
2  --RecordTypes va a ser igual a ese tipo de valores(puede variar)
3  data RecordType = Income | Expense | Saving | Investment
4  | deriving (Show, Read, Eq)
5
6  data FinancialRecord = FinancialRecord
7  { recordType :: RecordType
8  , amount     :: Double
9  , category   :: String
10 , date       :: String
11 , description :: String
12 , tags       :: [String]
13 } deriving (Show, Read)
14
15 -- Presupuesto máximo de gastos (Expense) para una categoría (nombre libre, se compara sin distinguir mayúsculas)
16 data Budget = Budget
17 { budgetCategory :: String
18 , budgetLimit    :: Double
19 } deriving (Show, Read, Eq)
20
21 -- Reglas configurables evaluadas sobre los registros cargados en memoria
22 data Rule
23 = -- Suma de gastos (Expense) en la categoría (normalizada) supera el umbral
24   RuleGastoEnCategoriaMayor String Double
25 | -- Suma de montos de ahorro (Saving) por debajo del mínimo deseado
26   RuleAhorroTotalMenor Double
```

Logic.hs

Contiene toda la lógica funcional del sistema.

En este módulo se implementaron:

- cálculos de balance
- análisis financieros
- simulaciones

- reglas
- reportes
- presupuestos

La lógica fue desarrollada utilizando funciones puras, composición de funciones, **map**, **filter**, **sum**, **Maybe**, **fold** y pattern matching.

Balance Funcional:

```

28 -- Indica si una categoría se pasó de su presupuesto.
29 excedePresupuesto :: Budget -> [FinancialRecord] -> Bool
30 excedePresupuesto b regs = gastoRealEnCategoría (budgetCategory b) regs > budgetLimit b
31
32 -- Genera mensajes de alerta para los presupuestos excedidos.
33 alertasPresupuesto :: [Budget] -> [FinancialRecord] -> [String]
34 alertasPresupuesto bs regs =
35   [ msg b
36   | b <- bs
37   , excedePresupuesto b regs
38   ]
39   where
40     msg b =
41       "ALERTA: categoría \""
42       ++ budgetCategory b
43       ++ "\" - gasto real "
44       ++ show (gastoRealEnCategoría (budgetCategory b) regs)
45       ++ " > presupuesto "
46       ++ show (budgetLimit b)
47

```

Reglas Funcionales:

```
67 totalAhorroRegistrado :: [FinancialRecord] -> Double
68 totalAhorroRegistrado regs =
69   | sum [amount r | r <- regs, recordType r == Saving]
70
71 -- Evalúa una regla individual y retorna mensaje si se activa.
72 evaluarRegla :: Rule -> [FinancialRecord] -> Maybe String
73 evaluarRegla (RuleGastoEnCategoriaMayor cat lim) regs =
74   | let g = gastoRealEnCategoria cat regs
75     in if g > lim
76       then
77         Just
78           ( "[ALERTA] Gastos en categoría \"
79             ++ cat
80             ++ "\" suman \"
81             ++ show g
82             ++ \" (supera el umbral \"
83             ++ show lim
84             ++ \")\"
85         )
86       else Nothing
87 evaluarRegla (RuleAhorroTotalMenor minimo) regs =
88   | let a = totalAhorroRegistrado regs
89     in if a < minimo
90       then
91         Just
92           ( "[ADVERTENCIA] Ahorro total registrado \"
93             ++ show a
94             ++ \" es menor al mínimo \"
95             ++ show minimo
96           )
97       else Nothing
98
99 -- Evalúa todas las reglas y junta solo los mensajes disparados.
100 evaluarReglas :: [Rule] -> [FinancialRecord] -> [String]
101 evaluarReglas rs regs = concatMap f rs
102   where
103     f r = case evaluarRegla r regs of
104       Nothing -> []
105       Just m -> [m]
```


Reportes/ Analisis:

```
114 -- Extrae el mes en formato YYYY-MM desde YYYY-MM-DD.
115 mesDeFecha :: String -> String
116 mesDeFecha f
117   | length f >= 7 = take 7 f
118   | otherwise = f
119
120 -- Lista de meses únicos y ordenados presentes en los registros.
121 mesesUnicosOrdenados :: [FinancialRecord] -> [String]
122 mesesUnicosOrdenados regs = sort (nub [mesDeFecha (date r) | r <- regs])
123
124 -- Flujo por mes: ingresos, gastos y neto.
125 resumenMensual :: [FinancialRecord] -> [(String, Double, Double, Double)]
126 resumenMensual regs =
127   [ (mes, ingresos, gastos, ingresos - gastos)
128   | mes <- meses
129   , let ingresos = sumaIncome mes
130   , let gastos = sumaExpense mes
131   ]
132   where
133     meses = mesesUnicosOrdenados regs
134     sumaIncome m =
135       sum [amount r | r <- regs, mesDeFecha (date r) == m, recordType r == Income]
136     sumaExpense m =
137       sum [amount r | r <- regs, mesDeFecha (date r) == m, recordType r == Expense]
138
139 -- Gasto total por mes (solo registros Expense).
140 gastosPorMes :: [FinancialRecord] -> [(String, Double)]
141 gastosPorMes regs =
142   [ (m, sum [amount r | r <- regs, recordType r == Expense, mesDeFecha (date r) == m])
143   | m <- mesesUnicosOrdenados regs
144   ]
145
146 -- Compara gasto de cada mes contra el mes anterior.
147 tendenciaGastoMensual :: [FinancialRecord] -> [(String, Double, Double, Double)]
148 tendenciaGastoMensual regs =
149   [ (mesActual, gastoActual, gastoPrevio, gastoActual - gastoPrevio)
150   | (_, gastoPrevio), (mesActual, gastoActual) <- zip gs (drop 1 gs)
151   ]
```

```
155 -- Proyección simple: promedio de gastos mensuales históricos.
156 proyeccionGastoSiguienteMes :: [FinancialRecord] -> Maybe Double
157 proyeccionGastoSiguienteMes regs =
158   case map snd (gastosPorMes regs) of
159     [] -> Nothing
160     xs -> Just (sum xs / fromIntegral (length xs))
161
162 -- Suma gasto por categoría para identificar impacto.
163 gastoPorCategoria :: [FinancialRecord] -> [(String, Double)]
164 gastoPorCategoria regs =
165   [ (c, sum [amount r | r <- regs, recordType r == Expense, category r == c])
166   | c <- categoriasUnicasGasto regs
167   ]
168
169 -- Top N categorías con mayor gasto acumulado.
170 topCategoriasGasto :: Int -> [FinancialRecord] -> [(String, Double)]
171 topCategoriasGasto n regs =
172   take n $
173     sortByGastoDesc
174       (gastoPorCategoria regs)
175
176 -- Lista categorías (de gastos) sin repetir.
177 categoriasUnicasGasto :: [FinancialRecord] -> [String]
178 categoriasUnicasGasto regs =
179   nub [category r | r <- regs, recordType r == Expense]
```

Storage.hs

Encargado de la persistencia de información mediante archivos de texto.

Este módulo permite:

- guardar registros
- cargar registros
- almacenar presupuestos
- almacenar reglas

Se utilizó serialización mediante **show** y reconstrucción mediante **read**.

```
1  module Storage where
2
3  import Types ( Budget, FinancialRecord, Rule )
4  import System.Directory (doesFileExist)
5
6  -- Guarda la lista de registros en un archivo de texto
7  -- Se usa show para serializar la lista
8  saveRecords :: [FinancialRecord] -> IO ()
9  saveRecords records = writeFile "records.txt" (show records)
10
11 saveBudgets :: [Budget] -> IO ()
12 saveBudgets bs = writeFile "budgets.txt" (show bs)
13
14 loadBudgets :: IO [Budget]
15 loadBudgets = do
16   exists <- doesFileExist "budgets.txt"
17   if not exists
18   then return []
19   else do
20     content <- readFile "budgets.txt"
21     return (read content)
22
23 -- Carga los registros desde el archivo
24 -- Si el archivo no existe, retorna lista vacía para evitar errores
25 loadRecords :: IO [FinancialRecord]
26 loadRecords = do
27   exists <- doesFileExist "records.txt"
28
29   if not exists
30   then return []
31   else do
32     content <- readFile "records.txt"
33     return (read content)
34
35 saveRules :: [Rule] -> IO ()
36 saveRules rs = writeFile "rules.txt" (show rs)
```

Main.hs

Contiene la interfaz de consola y el menú principal del sistema.

Este módulo administra:

- entrada de datos
- validaciones
- navegación del menú
- interacción con el usuario

La lógica principal se mantiene separada del IO para respetar el paradigma funcional.

Explicación del funcionamiento

El sistema ha sido desarrollado bajo un enfoque de programación funcional, utilizando el lenguaje Haskell. La arquitectura se basa en la separación de responsabilidades mediante módulos, garantizando que el estado del programa se maneje de forma efectiva y los datos sean manejables.

Procesamiento de Datos (Módulo Logic.hs):

Es el núcleo del sistema. Utiliza funciones de orden superior como filter, map y fold para procesar listas de registros financieros. Implementa la normalización de cadenas para que las comparaciones de categorías no fallen por diferencias en mayúsculas o espacios. Se encuentran los algoritmos que realizan los análisis y proyecciones de ahorro.

Persistencia de Información (Módulo Storage.hs):

La interacción con el disco se gestiona mediante mónadas de **I/O**. El sistema almacena la información en archivos de texto plano (records.txt, budgets.txt). Se utilizan las funciones read y show para la serialización de tipos de datos algebraicos, permitiendo que las estructuras complejas se recuperen íntegramente al reiniciar la aplicación. Incluye validaciones de seguridad para verificar la existencia de los archivos antes de intentar leerlos.

Interfaz y Control (Módulo Main.hs):

Implementa un menú recursivo que captura las entradas del usuario. La robustez del sistema se apoya en funciones de validación que utilizan readMaybe para filtrar datos erróneos (como montos negativos o fechas inválidas) sin que el programa se detenga.

Flujo de Trabajo:

Al ejecutarse, el programa carga los datos a memoria en estructuras de listas. Cada vez que el usuario añade un gasto o ingreso, el sistema genera una nueva versión de la lista y actualiza el archivo físico de forma inmediata. Para los reportes, el sistema realiza "vistas" dinámicas sobre los datos cargados, aplicando filtros temporales o por categoría según lo solicitado.

Bibliografía

A Gentle Introduction to Haskell: IO. (s. f.). <https://www.haskell.org/tutorial/io.html>

Cardiff, B. J. (2025, 24 marzo). *¿Cómo instalar Haskell?* DEV Community.

<https://dev.to/bcardiff/como-instalar-haskell-40ml>

Tutorial de Haskell | ¡Aprende como programar en Haskell! (2020, 14 octubre). Ionos Digital

Guide.

<https://www.ionos.com/es-us/digitalguide/paginas-web/desarrollo-web/tutorial-de-haskell/>